

API GATEWAY

with Go, Redis & Docker

A Complete Build-From-Scratch Guide
Covering Architecture • Implementation Concepts • Design Decisions

Stack	Go · Redis · Docker · JWT
Difficulty	Intermediate to Advanced
Topics Covered	Microservices, Reverse Proxy, Auth, Rate Limiting
Purpose	Production-style backend system design

1. What Is an API Gateway?

An API Gateway is a server that acts as the single entry point for all client requests into a system made up of multiple backend services. Think of it as the front door of a building — every visitor must pass through it, and it decides who can enter, where they go, and logs their visit.

In a microservices architecture, you have many small, independent services each doing one thing well. Without a gateway, clients would need to know the address of every service and call them directly. That creates chaos — authentication is duplicated, there's no central logging, and scaling becomes difficult.

The API Gateway solves this by centralising cross-cutting concerns:

- Authentication — verify who the caller is, once, in one place
- Rate limiting — prevent abuse, enforce fair usage policies
- Routing — forward requests to the right backend service

- Logging — record every request for monitoring and debugging
- Load balancing (advanced) — distribute load across service instances

1.1 The Problem It Solves

Imagine you have three services: user-service, product-service, and order-service. Without a gateway:

Without Gateway	With Gateway
Client calls user-service directly on port 8001	Client calls gateway on port 8080 only
Client calls product-service on port 8002	Gateway routes to the right service internally
Auth logic duplicated in every service	Auth happens once at the gateway
Rate limiting must be in each service	Rate limiting centralised in gateway + Redis
Hard to add logging uniformly	Single place to log all traffic

2. Architecture Overview

The system has four main layers that a request flows through. Understanding this flow is the foundation of everything else in the project.

CLIENT	External consumers (browsers, mobile apps, CLI tools, third-party services)
↓ ↑	HTTP/HTTPS requests and responses
API GATEWAY	Single entry point: auth → rate-limit → log → proxy
↓ routes	URL path-based routing to appropriate service
MICROSERVICES	user-service · product-service · order-service
← →	Gateway communicates with Redis for rate-limit state
REDIS	Token bucket state store — shared across all gateway instances

2.1 Request Lifecycle

Every single request — from the first byte arriving to the final response sent — follows this exact path through the gateway:

1**Request arrives at the Gateway**

A client sends an HTTP request to port 8080 (the gateway's public port). This is the only port exposed to the outside world.

2**JWT Authentication Middleware runs**

The gateway extracts the Authorization header, decodes the JWT token, verifies the signature using a secret key, and checks expiry. If this fails, it returns 401 Unauthorized immediately — the request never reaches a service.

3**Rate Limiting Middleware runs**

The gateway identifies the requester (by user ID from the JWT or by IP address), checks their current request count in Redis, and either allows or blocks the request. A blocked request gets a 429 Too Many Requests response.

4**Logging Middleware runs**

The gateway records the method, path, requester identity, and timestamp. Response logging (status code, latency) happens after the proxied response returns.

5**Reverse Proxy forwards the request**

Based on the URL path (e.g. /users/* maps to user-service), the gateway forwards the request to the correct microservice's internal address. It relays the response back to the client.

6**Response returns to Client**

The microservice's response travels back through the gateway (logging middleware captures status + latency) and is sent to the original client.

3. Core Concepts Deep Dive

3.1 Reverse Proxy

A reverse proxy is a **server that forwards requests on behalf of clients to one or more backend servers**. The client only knows about the proxy — it never communicates directly with the backend.

How Reverse Proxy Works in This Project

1. Client sends: GET /products/42 to gateway (port 8080)
2. Gateway inspects the path prefix: /products/*
3. Gateway looks up its routing table: /products → product-service:8002
4. Gateway creates a new internal request to product-service:8002/products/42
5. product-service responds with product data
6. Gateway relays that response back to the client

The client never knows product-service exists. Its address is hidden behind the gateway.

In Go, the standard library's `httputil.ReverseProxy` handles most of this. You configure a `Director` function that modifies the incoming request to point at the target service. The key pieces to configure are:

- Target URL — where to forward (e.g. `http://product-service:8002`)
- Path rewriting — strip or keep the prefix (e.g. `/products/42` might become `/42`)
- Header forwarding — pass along headers like `Authorization`, `X-Request-ID`
- Error handling — what to do when a backend service is down

3.2 JWT Authentication

JSON Web Tokens (JWT) are a compact, URL-safe way to represent claims between two parties. In this project, JWTs are used to verify that the caller is who they say they are, without hitting a database on every request.

JWT Structure

A JWT has three parts separated by dots: `header.payload.signature`

HEADER: algorithm and token type

```
{ "alg": "HS256", "typ": "JWT" }
```

PAYLOAD: claims (data about the user)

```
{ "user_id": "u_123", "role": "user", "exp": 1720000000 }
```

SIGNATURE: HMACSHA256(base64(header) + '.' + base64(payload), secret)

The signature proves the token was created by someone who knows the secret key.

If anyone tampers with the payload, the signature won't match → rejected.

The gateway's authentication middleware does the following on every request:

1. Extract the token from the Authorization: Bearer <token> header
2. Decode the Base64 header and payload (this requires no secret)
3. Re-compute the signature using the stored secret key
4. Compare computed signature with the one in the token
5. Check the exp (expiry) claim — reject if token is expired
6. If valid, extract user_id and attach it to the request context
7. Pass the request on to the next middleware (rate limiter)

What the Middleware Does NOT Do

It does NOT hit a database to validate the token — JWTs are self-contained.

It does NOT check if the user account still exists or is active.

It does NOT issue tokens — that's the responsibility of your auth/login service.

For this project scope, token issuance can be a simple helper function that generates a signed JWT when given a user ID and secret. In production, a dedicated auth service would handle this.

3.3 Rate Limiting with the Token Bucket Algorithm

Rate limiting prevents a single user or IP from overwhelming your system. The Token Bucket algorithm is one of the most common approaches. Here is how it works conceptually:

Token Bucket Mental Model

Imagine each user has a bucket that holds a maximum of N tokens.

- Tokens are added to the bucket at a constant rate (e.g. 10 tokens per minute)

- Each request consumes one token from the bucket
 - If the bucket is empty, the request is rejected (429 Too Many Requests)
 - Tokens accumulate when the user makes fewer requests than the refill rate
 - The bucket has a maximum capacity — it cannot overflow beyond N tokens
- This naturally allows short bursts (using accumulated tokens) while enforcing a long-term average rate limit. It's fairer than a hard per-minute counter.

The algorithm involves these fields stored per user in Redis:

Redis Field	Purpose
tokens	Current number of available tokens in the bucket
last_refill	Unix timestamp of the last token refill operation
max_tokens	Maximum bucket capacity (configured per user or globally)
refill_rate	How many tokens are added per second/minute

Why Redis for rate limiting? Because your gateway might run as multiple instances (horizontal scaling). If each instance tracked counts in memory, the limits would be per-instance, not per-user globally. Redis provides a shared, atomic state store that all gateway instances read from and write to.

Critical detail: The Redis operations (read tokens, compute new value, write back) must be atomic to avoid race conditions. Use Redis Lua scripts or the MULTI/EXEC transaction block to make this safe.

4. Project Structure

Before writing any code, it helps to design the folder structure. Here is a recommended layout:

Path	What lives here
api-gateway/	Root of the gateway service

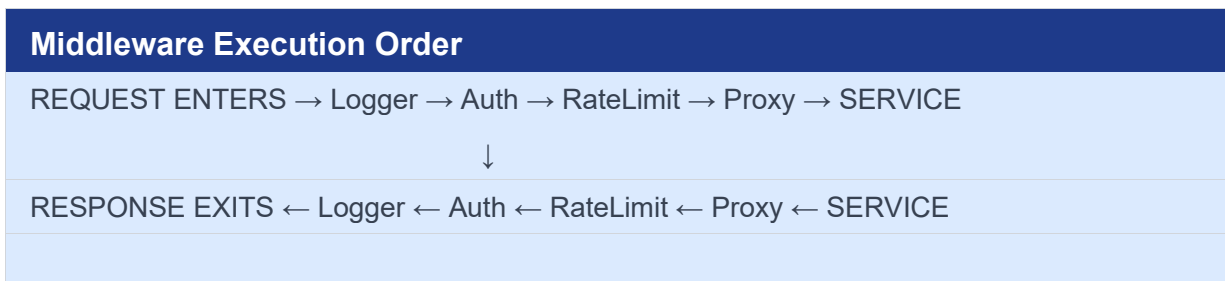
api-gateway/main.go	Entry point — starts the HTTP server
api-gateway/middleware/auth.go	JWT validation middleware
api-gateway/middleware/ratelimit.go	Token bucket rate limiter
api-gateway/middleware/logger.go	Request/response logging
api-gateway/proxy/proxy.go	Reverse proxy logic and route config
api-gateway/config/config.go	Environment config (ports, secrets, Redis URL)
user-service/	Stub user microservice
user-service/main.go	Simple HTTP server with /users endpoints
product-service/	Stub product microservice
product-service/main.go	Simple HTTP server with /products endpoints
docker-compose.yml	Orchestrates all containers together
gateway.Dockerfile	Container definition for the gateway
.env	Environment variables (never commit to git)

5. The Middleware Chain

Middleware is software that intercepts a request before it reaches its final handler. In Go, middleware is implemented as a function that wraps an `http.Handler`. Each middleware does its job, then either calls the next middleware or returns an error response.

5.1 How Middleware Chaining Works

Think of middleware as layers of an onion. The request enters from the outside, passes through each layer inward, reaches the core (reverse proxy), then the response travels back out through the layers in reverse:



On the way IN: Logger starts timer, Auth validates token, RateLimit checks bucket

On the way OUT: Logger records status code and latency

Important: Auth runs before RateLimit because we need the user identity
(from the JWT) to know which bucket to check in Redis.

5.2 Logger Middleware

The logger middleware captures information about every request. It records the data before forwarding, then wraps the `ResponseWriter` to capture the status code after the response is written.

What to log:

- Timestamp — when the request arrived
- HTTP Method — GET, POST, PUT, DELETE, etc.
- URL Path — which endpoint was called
- Client IP — for debugging and security analysis
- Response Status Code — 200, 401, 429, 500, etc.
- Latency — how long the full round-trip took in milliseconds
- User ID — extracted from JWT (if available)

5.3 Auth Middleware

This middleware runs after logging starts. It must succeed before the request proceeds. Implementation steps in Go:

8. Read the Authorization header. Return 401 if missing.
9. Check it starts with 'Bearer '. Return 401 if not.
10. Extract the token string after 'Bearer '.
11. Parse and validate the token using the JWT library (jwt-go or golang-jwt).
12. Check the signing method is what you expect (HS256). Return 401 if not.
13. Verify the token is not expired (check the exp claim).
14. Extract the user ID from the claims and store in request context.
15. Call next handler to continue the chain.

5.4 Rate Limit Middleware

This middleware runs after authentication, so the user identity is available in the context. It uses Redis to implement a distributed token bucket:

16. Get user ID from request context (set by auth middleware).
17. Construct a Redis key: 'ratelimit:user:<user_id>'.
18. Use a Lua script (atomic) to: read current tokens and last refill time, compute how many tokens to add based on elapsed time, add them (up to max capacity), deduct one token for this request, write the new state back.
19. If tokens were available: allow the request, set X-RateLimit-Remaining header.
20. If no tokens: return 429 Too Many Requests with Retry-After header.

6. Docker and Docker Compose

6.1 Why Docker?

Docker packages your application and all its dependencies into a self-contained image. When you run the image as a container, it behaves identically regardless of the host machine. For this project, Docker solves these problems:

- Each service (gateway, user-service, product-service, Redis) runs in isolation
- No 'works on my machine' problems — same image, same behaviour everywhere
- Services can communicate over a private Docker network without exposing ports publicly
- Easy to scale: run multiple instances of a service with one command

6.2 Docker Compose Concepts

Docker Compose is a tool for defining and running multi-container applications. You define everything in a docker-compose.yml file:

Compose Concept	What It Means for This Project
services	Each container is a service: gateway, user-service, product-service, redis
build	Path to the Dockerfile for building the service's image
ports	Maps host port to container port (e.g. '8080:8080' exposes gateway)

environment	Environment variables passed into the container (JWT secret, Redis URL)
depends_on	Ensures redis starts before gateway, gateway starts after services
networks	All services share a private network — they can reach each other by name
volumes	Persist Redis data outside the container (survives restarts)

6.3 Networking in Docker Compose

This is a concept many beginners miss. When services run in Docker Compose, they can reference each other by their service name as a hostname. For example:

- The gateway can connect to Redis at redis:6379 (not localhost:6379)
- The gateway can forward requests to user-service at http://user-service:8001
- From outside Docker, only the gateway's port 8080 is accessible

This means your routing config in the gateway uses internal Docker network addresses, not localhost. This is a common source of confusion when first building the project.

7. Build Order and Milestones

Building this project in the right order prevents frustration. Each milestone is independently testable before moving on.

Milestone 1 — Basic HTTP Server in Go

Goal: A Go server that responds to HTTP requests on port 8080.

Test: curl http://localhost:8080/health returns 200 OK.

Concepts: net/http package, http.HandleFunc, http.ListenAndServe, graceful shutdown.

Do NOT add middleware yet. Get the basic server working first.

Milestone 2 — Simple Reverse Proxy

Goal: Forward requests from gateway to a hardcoded backend (stub service).

Test: `curl http://localhost:8080/users` returns response from user-service.

Concepts: `httputil.NewSingleHostReverseProxy`, `Director` function, URL path rewriting.

Build the stub services first (simple Go HTTP servers), then connect the proxy.

Milestone 3 — Dynamic Routing

Goal: Route based on path prefix to different services.

Test: `/users/*` goes to user-service, `/products/*` goes to product-service.

Concepts: Route table (map of prefix → target URL), path matching with `strings.HasPrefix`.

Return 404 if no route matches.

Milestone 4 — Logger Middleware

Goal: Log every request with method, path, status, and latency.

Test: Make requests and see structured log output in the terminal.

Concepts: Middleware pattern in Go, custom `ResponseWriter` to capture status code, `time.Now()` and `time.Since()` for latency measurement.

Milestone 5 — JWT Authentication

Goal: Reject requests without a valid JWT. Allow requests with a valid one.

Test: `curl` without token → 401. `curl` with valid token → 200.

Concepts: `jwt-go` or `golang-jwt` library, HMAC signing, `context.WithValue` to pass user ID.

Write a helper function to generate test tokens for manual testing.

Milestone 6 — Redis Rate Limiting

Goal: Enforce per-user request limits using token bucket in Redis.

Test: Send 15 requests rapidly (limit = 10/min) → first 10 succeed, next 5 get 429.

Concepts: `go-redis` library, Lua scripting for atomic operations, `X-RateLimit` headers.

Run Redis locally with Docker: `docker run -p 6379:6379 redis`

Milestone 7 — Full Docker Compose Setup

Goal: `docker-compose up` starts everything and the system works end-to-end.

Test: All milestones work when running fully containerised.

Concepts: Dockerfile for each service (multi-stage builds for small images),
docker-compose.yml with service dependencies, environment variable injection,
container networking (service names as hostnames).

8. Key Design Decisions

8.1 Why Go?

Go is an excellent choice for an API gateway because:

- Built-in HTTP server — `net/http` is production-ready without external frameworks
- Goroutines — lightweight concurrency model handles thousands of concurrent connections
- Compiled binary — fast startup, low memory footprint, easy to containerise
- Standard library — `httputil.ReverseProxy` is built in, no third-party proxy library needed
- Static typing — catches bugs at compile time, easier to maintain

You can use either the standard library (`net/http`) or the Gin framework. For learning, the standard library teaches more fundamentals. Gin adds routing conveniences and reduces boilerplate.

8.2 Why Redis for Rate Limiting (not in-memory)?

Storing rate limit state in-memory would work for a single-instance gateway. However, it breaks immediately when you run multiple gateway instances for reliability or scalability. Redis provides:

- Shared state — all gateway instances read and write the same counters
- Atomicity — Redis Lua scripts ensure read-modify-write operations are atomic
- TTL (Time To Live) — Redis keys can auto-expire, cleaning up stale user buckets
- Performance — Redis operates in memory with microsecond latency

8.3 Middleware Order Matters

The order `Logger` → `Auth` → `RateLimit` → `Proxy` is deliberate:

- `Logger` first: starts the timer before anything else, captures all outcomes including auth failures
- `Auth` before `RateLimit`: rate limiting is per-user, so we need the user ID from the JWT first

- RateLimit before Proxy: no point forwarding to a service if we're going to reject the request anyway
- Proxy last: only reached if all previous middleware passed

8.4 Not Storing Tokens in Blocklists

A common question is: how do you invalidate a JWT before it expires? The short answer is: you need a blocklist, but this project does not implement one. In production, you would store revoked token IDs (jti claim) in Redis with TTL equal to their remaining validity. For this project, token expiry is relied upon entirely.

9. Configuration Management

Hard-coding values (ports, secrets, URLs) directly in code is a bad practice. Instead, use environment variables. This allows the same code to run in development, staging, and production with different configs.

Environment Variable	Purpose
GATEWAY_PORT	Port the gateway listens on (default: 8080)
JWT_SECRET	Secret key used to sign and verify JWTs (keep this safe!)
REDIS_URL	Redis connection string (e.g. redis:6379 in Docker)
USER_SERVICE_URL	Internal URL for user-service (e.g. http://user-service:8001)
PRODUCT_SERVICE_URL	Internal URL for product-service (e.g. http://product-service:8002)
RATE_LIMIT_MAX_TOKENS	Token bucket capacity per user (e.g. 100)
RATE_LIMIT_REFILL_RATE	Tokens added per second per user (e.g. 10)
LOG_LEVEL	Logging verbosity: debug, info, warn, error

Security Warning

NEVER commit your .env file to git. Add it to .gitignore immediately.

NEVER hardcode JWT_SECRET in source code.

Use a .env.example file with placeholder values to show what variables are needed.

In production, use a secrets manager (AWS Secrets Manager, HashiCorp Vault, etc.) instead of environment variables in plain text.

10. Testing Strategy

10.1 Manual Testing with curl

During development, curl is your best friend. Here are the commands you'll use constantly:

```
# Test without token (expect 401)
curl -i http://localhost:8080/users
```

```
# Generate a test token (you write this helper)
go run cmd/gen_token/main.go --user=u_123 --secret=mysecret
```

```
# Test with valid token (expect 200)
curl -i -H 'Authorization: Bearer <token>' http://localhost:8080/users
```

```
# Test rate limiting (run in a loop, expect 429 after N requests)
for i in $(seq 1 20); do curl -s -o /dev/null -w '%{http_code}\n' -H
'Authorization: Bearer <token>' http://localhost:8080/users; done
```

10.2 Unit Tests

Write unit tests for the middleware functions. Test each middleware in isolation by creating a mock `http.Handler` as the 'next' handler and a mock `http.ResponseWriter` to capture the response.

Test Case	Expected Outcome
Auth middleware: no Authorization header	Returns 401 Unauthorized
Auth middleware: malformed Bearer token	Returns 401 Unauthorized
Auth middleware: expired token	Returns 401 Unauthorized
Auth middleware: valid token	Calls next handler, user ID in context

Rate limiter: bucket has tokens	Calls next handler, decrements token count
Rate limiter: bucket is empty	Returns 429 Too Many Requests
Logger middleware: any request	Logs method, path, status, latency
Proxy: known route /users/*	Forwards to user-service URL
Proxy: unknown route /unknown/*	Returns 404 Not Found

11. Common Pitfalls and How to Avoid Them

Pitfall 1 — Connecting to localhost inside Docker

WRONG: Redis URL = localhost:6379

RIGHT: Redis URL = redis:6379 (Docker Compose service name)

When a service runs inside a container, 'localhost' is the container itself, not the host machine. Use the service name defined in docker-compose.yml.

Pitfall 2 — Race condition in rate limiting

WRONG: Read tokens from Redis → compute new value → write back (three separate ops)

RIGHT: Use a Lua script so all three operations are atomic

Without atomicity, two concurrent requests can both read '1 token remaining', both proceed, and both write back '0 tokens' — but you served two requests instead of one, violating the rate limit.

Pitfall 3 — Not stripping the path prefix

WRONG: Forward /users/42 to user-service as /users/42

RIGHT: Forward /users/42 to user-service as /42 (if service expects /42)

Depends on how your microservices are designed. Some expect the full path,

some expect the prefix stripped. Configure the Director function accordingly.

Test this by hitting the microservice directly before connecting via gateway.

Pitfall 4 — Middleware order bugs

WRONG: RateLimit before Auth (rate limiter has no user ID to key on)

RIGHT: Auth → RateLimit (JWT sets user ID in context, rate limiter reads it)

If rate limiting runs before auth, you either can't do per-user limiting (only per-IP) or you'll get a nil user ID panic. Always authenticate first.

Pitfall 5 — ResponseWriter already written error

If a middleware calls `http.Error()` or writes a response AND then calls `next()` handler which also writes a response, you'll get a 'superfluous response.WriteHeader' warning and broken responses.

Pattern: Always return after writing an error response. Never call `next()` after writing an error. Use a boolean to track whether a response has been sent.

12. Extensions and Next Steps

Once the core project works, here are extensions to make it more production-like. These are excellent talking points in interviews and vivas.

Extension	What It Adds / Teaches
Health check endpoint	GET /health returns 200 — used by load balancers and Docker to verify service is alive
Request ID tracing	Generate a UUID per request, pass it as X-Request-ID header to all services for distributed tracing
Circuit breaker	If a backend service is repeatedly failing, stop sending it traffic for a cooldown period (prevents cascade failures)
Load balancing	Maintain a list of instances per service, round-robin or least-connections selection

Token blocklist	Store revoked JWT IDs in Redis to enable instant token invalidation
Prometheus metrics	Export request count, latency histograms, error rates to Prometheus for Grafana dashboards
HTTPS/TLS termination	Handle SSL at the gateway, forward plain HTTP internally
Response caching	Cache identical GET responses in Redis for a configurable TTL
API versioning	Route /v1/* and /v2/* to different service versions
Graceful shutdown	Handle SIGTERM: stop accepting new requests, wait for in-flight requests to finish

Glossary

Term	Definition
API Gateway	A server that acts as the single entry point for all client requests to microservices
Reverse Proxy	A server that forwards requests on behalf of clients to backend servers
JWT	JSON Web Token — a signed, self-contained token encoding user claims
Token Bucket	A rate-limiting algorithm using a metaphorical bucket of tokens, refilled over time
Middleware	A function that intercepts and processes requests before they reach the final handler
Microservice	A small, independently deployable service focused on a single business capability
Redis	An in-memory data store used here for rate limit state (token bucket counters)
Docker Compose	A tool for defining and running multi-container Docker applications
Container	A lightweight, isolated runtime environment for a service and its dependencies
HTTP Handler	In Go, a type implementing <code>ServeHTTP(ResponseWriter, *Request)</code>
Goroutine	A lightweight Go thread — Go's concurrency primitive
Atomic operation	An operation that completes without interruption — used in rate limiting to prevent race conditions

Director function	In Go's ReverseProxy, the function that modifies the request before forwarding it
Context	In Go HTTP, a per-request object used to pass values (like user ID) between middleware
Environment variable	A runtime configuration value passed to a process from its environment
TLS/HTTPS	Encrypted HTTP — handles at the gateway; services communicate over plain HTTP internally

Build it step by step. Test each milestone before moving on. Read the Go docs.